Lab Sheet 2 — NoSQL Database Design

Q1. MySQL and MongoDB Student Database

```sql
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE Student (
 Student_ID INT PRIMARY KEY,
 Name VARCHAR(50),
 Course VARCHAR(30),
 Semester INT,
 Marks INT
);
INSERT INTO Student VALUES
(1,'Aarav','BCA',5,85),
(2,'Diya','BCA',5,90),
(3,'Kabir','BSc CS',4,78),
(4,'Meera','BCA',3,88),
(5,'Rohan','BCom',2,76);
SELECT * FROM Student;
```

```javascript
use CollegeDB
db.Students.insertMany([
{Student_ID:1,Name:"Aarav",Course:"BCA",Semester:5,Marks:85},
{Student_ID:2,Name:"Diya",Course:"BCA",Semester:5,Marks:90},
```

{Student_ID:3,Name:"Kabir",Course:"BSc CS",Semester:4,Marks:78},

{Student_ID:4,Name:"Meera",Course:"BCA",Semester:3,Marks:88},

{Student_ID:5,Name:"Rohan",Course:"BCom",Semester:2,Marks:76}

])

db.Students.find({})

```

Comparison: MySQL stores data in tables with a fixed schema. MongoDB stores flexible BSON documents in collections. MongoDB supports horizontal scaling through sharding.

### Q2. MongoDB CRUD Operations

```javascript

use CollegeDB

db.Students.insertMany([

{Student_ID:1,Name:"Aarav",Course:"BCA",Semester:5,Marks:85},

{Student_ID:2,Name:"Diya",Course:"BCA",Semester:5,Marks:90},

{Student_ID:3,Name:"Kabir",Course:"BSc CS",Semester:4,Marks:78},

{Student_ID:4,Name:"Meera",Course:"BCA",Semester:3,Marks:88},

{Student_ID:5,Name:"Rohan",Course:"BCom",Semester:2,Marks:76}

])

db.Students.find({})

db.Students.find({Course:"BCA"})

db.Students.updateOne({Student_ID:1},{$set:{Marks:95}})

db.Students.deleteOne({Student_ID:5})

db.Students.countDocuments({})

```

Q3. MongoDB Orders Collection
```

```javascript

db.Orders.insertMany([

{Order_ID:1,Customer:"Aarav",Products:[{Name:"Laptop",Price:50000,Qty:1}]},

{Order_ID:2,Customer:"Diya",Products:[{Name:"Phone",Price:20000,Qty:1}]},

{Order_ID:3,Customer:"Kabir",Products:[{Name:"Mouse",Price:500,Qty:2}]},

{Order_ID:4,Customer:"Meera",Products:[{Name:"Monitor",Price:12000,Qty:1}]},

{Order_ID:5,Customer:"Rohan",Products:[{Name:"Tablet",Price:15000,Qty:1}]}

])

db.Orders.find({Customer:"Aarav"})

db.Orders.updateOne(

{Order_ID:3},

{$set:{"Products.0.Qty":3}}

)

db.Orders.aggregate([

{project:{Order\_ID:1,Total:{sum:{$map:{

input:"$Products",as:"p",

in:{multiply:&#91;"$p.Price", "$p.Qty"]}

}}}}}

])

db.Orders.aggregate([

{addFields:{Total:{sum:{$map:{

input:"$Products",as:"p",

in:{multiply:&#91;"$p.Price", "$p.Qty"]}

}}}}},

{match:{Total:{gt:10000}}}
```

])

```

### Q4. Student Information System Using Redis

```text

SET 101 "Aarav"

SET 102 "Diya"

SET 103 "Kabir"

SET 104 "Meera"

SET 105 "Rohan"

GET 101

SET 101 "Aarav Sharma"

DEL 105

KEYS *

```

Q5. Cassandra Students Table

```sql

CREATE KEYSPACE CollegeDB

WITH replication = {'class':'SimpleStrategy','replication_factor':1};

USE CollegeDB;

CREATE TABLE Students (

Student_ID int PRIMARY KEY,

Name text,

Course text,

Semester int,

Marks int

```
);

INSERT INTO Students VALUES (1,'Aarav','BCA',5,85);

INSERT INTO Students VALUES (2,'Diya','BCA',5,90);

INSERT INTO Students VALUES (3,'Kabir','BSc CS',4,78);

INSERT INTO Students VALUES (4,'Meera','BCA',3,88);

INSERT INTO Students VALUES (5,'Rohan','BCom',2,76);

SELECT * FROM Students;

SELECT * FROM Students WHERE Course='BCA' ALLOW FILTERING;

UPDATE Students SET Marks=95 WHERE Student_ID=1;

DELETE FROM Students WHERE Student_ID=5;
```

Q6. HBase Students Table

```text
create 'Students','personal','academic'

put 'Students','101','personal:Name','Aarav'

put 'Students','101','academic:Course','BCA'

put 'Students','101','academic:Marks','85'

put 'Students','102','personal:Name','Diya'

put 'Students','103','personal:Name','Kabir'

put 'Students','104','personal:Name','Meera'

put 'Students','105','personal:Name','Rohan'

get 'Students','101'

put 'Students','101','academic:Marks','95'

scan 'Students'

deleteall 'Students','105'
```

```

### Q7. Neo4j Student–Course Graph

```cypher

CREATE

(s1:Student {id:1,name:'Aarav'}),

(s2:Student {id:2,name:'Diya'}),

(s3:Student {id:3,name:'Kabir'}),

(s4:Student {id:4,name:'Meera'}),

(s5:Student {id:5,name:'Rohan'}),

(c1:Course {name:'BCA'}),

(c2:Course {name:'BSc CS'}),

(c3:Course {name:'BCom'}),

(s1)-[:ENROLLED_IN]->(c1),

(s2)-[:ENROLLED_IN]->(c1),

(s3)-[:ENROLLED_IN]->(c2),

(s4)-[:ENROLLED_IN]->(c1),

(s5)-[:ENROLLED_IN]->(c3);

MATCH (s:Student) RETURN s;

MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {name:'BCA'})

RETURN s.name;

MATCH (:Student {name:'Aarav'})-[:ENROLLED_IN]->(c:Course)

RETURN c.name;

MATCH (s:Student {name:'Aarav'}),(c:Course {name:'BSc CS'})

CREATE (s)-[:ENROLLED_IN]->(c);

MATCH (:Student {name:'Aarav'})-[r:ENROLLED_IN]->(:Course {name:'BSc CS'})
```
```

DELETE r;

```

### Q8. Compare MongoDB, Cassandra, and Neo4j

```text

MongoDB:

Model: Document

Storage: Collections and BSON documents

Query: MongoDB Query Language

Scalability: Sharding

Use: Web applications and content management

Cassandra:

Model: Wide-column

Storage: Tables and partitions

Query: CQL

Scalability: Distributed horizontal scaling

Use: Large-scale distributed data

Neo4j:

Model: Graph

Storage: Nodes and relationships

Query: Cypher

Scalability: Depends on deployment

Use: Social networks and recommendation systems

```

Q9. MongoDB Local CRUD Operations

```javascript

use LocalDB

db.Students.insertMany([

{id:1,name:"Aarav",marks:85},

{id:2,name:"Diya",marks:90},

{id:3,name:"Kabir",marks:78}

])

db.Students.find({})

db.Students.updateOne({id:1},{$set:{marks:95}})

db.Students.deleteOne({id:3})

db.Students.find({})

db.Students.countDocuments({})

```

Q10. MongoDB Sharding

Requires a configured sharded cluster and connection through `mongos`.

```javascript

sh.enableSharding("ShardDB")

use ShardDB

for(let i=1;i<=20;i++){

db.Students.insertOne({

Student_ID:i,

Name:"Student"+i,

Marks:70+i%30

})

}

db.Students.createIndex({Student_ID:"hashed"})

```
sh.shardCollection(

"ShardDB.Students",

{Student_ID:"hashed"}

)

sh.status()

db.Students.getShardDistribution()
```

Benefit: Sharding distributes data across multiple servers and improves scalability.

Q11. Primary–Secondary Replication

Requires two running MongoDB instances configured as a replica set.

```javascript
rs.initiate({

_id:"rs0",

members:[

{_id:0,host:"localhost:27017"},

{_id:1,host:"localhost:27018"}

]

})

rs.status()

use ReplicationDB

db.Students.insertOne({

id:1,name:"Aarav",course:"BCA"

})
```

Result: The primary accepts writes, and secondary members replicate data. An eligible secondary may become primary if election requirements are met.

Q12. Cassandra Peer-to-Peer Replication

Requires a running multi-node Cassandra cluster.

```sql
CREATE KEYSPACE CollegeDB
WITH replication = {
'class':'NetworkTopologyStrategy',
'datacenter1':3
};
USE CollegeDB;
CREATE TABLE Students (
Student_ID int PRIMARY KEY,
Name text,
Course text,
Marks int
);
INSERT INTO Students VALUES (1,'Aarav','BCA',85);
INSERT INTO Students VALUES (2,'Diya','BCA',90);
SELECT * FROM Students;
```

```text
nodetool status
```

Result: Cassandra nodes communicate as peers and replicate data according to the configured replication factor.

Q13. Sharding with Replication

```text

Architecture:

Application

    |

  mongos

  / \

Shard 1 Shard 2

(Replica Set) (Replica Set)

 Primary Primary

 Secondary Secondary

```

```javascript

sh.enableSharding("CollegeDB")

use CollegeDB

for(let i=1;i<=20;i++){

db.Students.insertOne({

Student_ID:i,

Name:"Student"+i,

Marks:70+i%30

})

}

db.Students.createIndex({Student_ID:"hashed"})

sh.shardCollection(

"CollegeDB.Students",
```

{Student_ID:"hashed"}

)

sh.status()

db.Students.getShardDistribution()

```

Result: Sharding improves scalability, while replication provides fault tolerance.

### Q14. MapReduce in MongoDB

```javascript

db.StudentMarks.insertMany([

{Student_ID:1,Name:"Aarav",Course:"BCA",Marks:85},

{Student_ID:2,Name:"Diya",Course:"BCA",Marks:90},

{Student_ID:3,Name:"Kabir",Course:"BSc CS",Marks:78},

{Student_ID:4,Name:"Meera",Course:"BCA",Marks:88},

{Student_ID:5,Name:"Rohan",Course:"BSc CS",Marks:80}

])

var mapFunction = function(){

emit(this.Course,{count:1,total:this.Marks});

};

var reduceFunction = function(course,values){

var result={count:0,total:0};

values.forEach(function(v){

result.count+=v.count;

result.total+=v.total;

});

return result;

```
};

db.StudentMarks.mapReduce(

mapFunction,

reduceFunction,

{out:"CourseStatistics"}

);

db.CourseStatistics.find({});
```

Expected results:

```text

BCA: Count = 3, Average = 87.67

BSc CS: Count = 2, Average = 79
```

Note: `mapReduce()` is deprecated in modern MongoDB versions; aggregation is the recommended alternative.

Q15. Database Distribution Models

```text

1. SINGLE SERVER

Architecture: Application → Database

Distribution: All data on one server.

Failure: Server failure makes data unavailable.

Advantage: Simple setup.

Limitation: Limited scalability.

Application: Small college database.

2. SHARDING
```

Architecture: Application → Router → Multiple shards.

Distribution: Data divided among shards.

Failure: A shard failure can affect availability.

Advantage: Horizontal scalability.

Limitation: Complex configuration.

Application: E-commerce.

3. PRIMARY–SECONDARY REPLICATION

Architecture: Primary → Secondary replicas.

Distribution: Copies of data on multiple servers.

Failure: Secondary may be elected primary.

Advantage: High availability.

Limitation: Replication lag.

Application: Banking support systems.

4. PEER-TO-PEER REPLICATION

Architecture: Node 1 ↔️ Node 2 ↔️ Node 3.

Distribution: Data replicated across peer nodes.

Failure: Other replicas may continue serving requests.

Advantage: No permanent central write-primary.

Limitation: Consistency management.

Application: Distributed Cassandra systems.

5. SHARDING + REP

LICATION

Architecture: Router → Shards, each with replicas.

Distribution: Data partitioned and replicated.

Failure: Replicas can maintain availability.

Advantage: Scalability and fault tolerance.

Limitation: Higher complexity and cost.

Application: Large online platforms.